```
import asyncio
import base64
import hashlib
import logging
import os
import time

from aiogram import Bot, Dispatcher, F
from aiogram.filters import Command, CommandObject, CommandStart
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.types import CallbackQuery, Message
from aiogram.utils.keyboard import InlineKeyboardBuilder, ReplyKeyboardBuilder
from bson import ObjectId
from cryptography.fernet import Fernet
from pymongo import AsyncMongoClient

logging.basicConfig(level=logging.INFO)


# ============================================================
#  SOZLAMALAR (Railway > Variables)
# ============================================================
def env_int(name, default=0):
    try:
        return int(os.getenv(name, default))
    except ValueError:
        return default


BOT_TOKEN = os.environ["BOT_TOKEN"]              # asosiy bot tokeni
MONGO_URI = os.environ["MONGO_URI"]              # MongoDB havolasi
ENCRYPT_SECRET = os.environ["ENCRYPT_SECRET"]    # tokenlarni shifrlash uchun maxfiy
matn
ADMIN_ID = env_int("ADMIN_ID")                   # sizning Telegram ID raqamingiz
BOT_PRICE = env_int("BOT_PRICE", 0)              # bitta bot yaratish narxi (0 =
bepul)
REFERAL_BONUS = env_int("REFERAL_BONUS", 0)      # taklif uchun bonus (so'm)
MAX_BOTS = env_int("MAX_BOTS", 10)               # bitta odam nechta bot yarata oladi
CARD_INFO = os.getenv("CARD_INFO", "")           # to'lov uchun karta raqami va ism

# ============================================================
#  BAZA VA SHIFRLASH
# ============================================================
mongo = AsyncMongoClient(MONGO_URI)
db = mongo["bot_yaratuvchi"]
```

```
bots = db["bots"]              # yaratilgan botlar
bot_users = db["bot_users"]    # yaratilgan botlarning foydalanuvchilari
accounts = db["accounts"]      # hisoblar (balans, referal)
topups = db["topups"]          # pul kiritish so'rovlari
movies = db["movies"]          # kino botlardagi fayllar
msg_map = db["msg_map"]        # javob berish uchun xabarlar xaritasi
settings = db["settings"]      # sozlamalar (karta matni)

fernet =
Fernet(base64.urlsafe_b64encode(hashlib.sha256(ENCRYPT_SECRET.encode()).digest()))

running = {}  # bot_id -> (bot, task)

# ========================================================
#  BOT TURLARI KATALOGI
#  engine: movies = kino/fayl boti, menu = tugmali bot,
#          contact = murojaat boti, builder = BuilderBot,
#          soon = tez orada
# ========================================================
HELLO = "Salom! Xush kelibsiz 👋"
MENU_BTNS = [
    {"title": "ℹ️ Biz haqimizda", "reply": "Bu yerga o'zingiz haqingizda yozasiz."},
    {"title": "📞 Aloqa", "reply": "Telefon: +998 ** *** ** **"},
]

TEMPLATES = {
    # Media
    "kino": {"title": "🎬 Kino bot", "engine": "movies", "welcome": "🎬 Kino kodini
yuboring va filmni oling!"},
    "kinopro": {"title": "🎬 Kinobot Pro", "engine": "movies", "welcome": "🎬 Kino
kodini yuboring!"},
    "multfilm": {"title": "🧸 Multfilm", "engine": "movies", "welcome": "🧸 Multfilm
kodini yuboring!"},
    "kinoilova": {"title": "🎬 Kino Ilova", "engine": "movies", "welcome": "🎬 Kino
kodini yuboring!"},
    "provideo": {"title": "📽️ Pro Video Bot", "engine": "movies", "welcome": "📽️
Video kodini yuboring!"},
    "drama": {"title": "🎭 DramaBot", "engine": "movies", "welcome": "🎭 Serial/drama
kodini yuboring!"},
    "audiopechat": {"title": "🎵 Audio Pechat Bot", "engine": "soon", "welcome":
HELLO},
    "yuklovchi": {"title": "⚡ Video va musiqa yuklovchi (yangi)", "engine": "soon",
"welcome": HELLO},
    # SMM
    "smm_xizmat": {"title": "📣 SMM xizmatlar boti", "engine": "menu", "welcome": "📣
SMM xizmatlarimizga xush kelibsiz!"},
    "smm_murojaat": {"title": "📩 Murojaat boti", "engine": "contact", "welcome": "📩
Savol yoki taklifingizni yozing, tez orada javob beramiz."},
    "smm_post": {"title": "📝 Avto-post boti", "engine": "soon", "welcome": HELLO},
    # Moliyaviy
```

```
    "fin_narx": {"title": "🧾 Xizmat narxlari boti", "engine": "menu", "welcome": "🧾 Xizmat narxlarimiz bilan tanishing:"},
    "fin_kurs": {"title": "💱 Valyuta kursi boti", "engine": "soon", "welcome": HELLO},
    # Savdo biznes
    "shop_katalog": {"title": "🛍️ Katalog boti", "engine": "menu", "welcome": "🛍️ Mahsulotlarimiz katalogiga xush kelibsiz!"},
    "shop_buyurtma": {"title": "🛒 Buyurtma qabul qilish boti", "engine": "contact", "welcome": "🛒 Buyurtmangizni yozing, operator siz bilan bog'lanadi."},
    # Ta'lim va tarjimon
    "edu_markaz": {"title": "📚 O'quv markaz boti", "engine": "menu", "welcome": "📚 O'quv markazimizga xush kelibsiz!"},
    "edu_tarjimon": {"title": "🌐 Tarjimon boti", "engine": "soon", "welcome": HELLO},
    # Obuna
    "sub_majburiy": {"title": "🎣 Majburiy obuna boti", "engine": "soon", "welcome": HELLO},
    "sub_vip": {"title": "👑 VIP obuna boti", "engine": "soon", "welcome": HELLO},
    # Guruh
    "grp_admin": {"title": "🛡️ Guruh admin boti", "engine": "soon", "welcome": HELLO},
    "grp_salom": {"title": "🙋 Guruh salomlashish boti", "engine": "soon", "welcome": HELLO},
    # Sun'iy intellekt
    "ai_chat": {"title": "🧠 AI suhbat boti", "engine": "soon", "welcome": HELLO},
    "ai_rasm": {"title": "🎨 AI rasm boti", "engine": "soon", "welcome": HELLO},
    # Foydali servis
    "util_info": {"title": "ℹ️ Ma'lumot boti", "engine": "menu", "welcome": "ℹ️ Kerakli bo'limni tanlang:"},
    "util_ob": {"title": "🌤️ Ob-havo boti", "engine": "soon", "welcome": HELLO},
    # Avtomatlashtirish
    "auto_javob": {"title": "🤖 Avto-javob boti", "engine": "menu", "welcome": "🤖 Savolingizni tanlang:"},
    "auto_eslatma": {"title": "⏰ Eslatma boti", "engine": "soon", "welcome": HELLO},
    # BuilderBot
    "builder": {"title": "🏗️ BuilderBot", "engine": "builder", "welcome": "Salom! Bu yerda o'z botingizni yaratishingiz mumkin 🤖"},
}

CATEGORIES = [
    ("🎬 Media botlar", ["kino", "kinopro", "multfilm", "kinoilova", "provideo", "drama", "audiopechat", "yuklovchi"]),
    ("🚀 SMM botlar", ["smm_xizmat", "smm_murojaat", "smm_post"]),
    ("💰 Moliyaviy botlar", ["fin_narx", "fin_kurs"]),
    ("🛍️ Savdo biznes botlar", ["shop_katalog", "shop_buyurtma"]),
    ("📚 Ta'lim va tarjimon botlar", ["edu_markaz", "edu_tarjimon"]),
    ("👑 Obuna botlar", ["sub_majburiy", "sub_vip"]),
    ("👥 Guruh uchun botlar", ["grp_admin", "grp_salom"]),
    ("💬 Sun'iy intellekt botlar", ["ai_chat", "ai_rasm"]),
    ("🧰 Foydali servis botlar", ["util_info", "util_ob"]),
```

```
    ("🔄 Avtomatlashtirish botlar", ["auto_javob", "auto_eslatma"]),
]

# Asosiy menyu tugmalari
B_CREATE = "➕ Bot yaratish"
B_MY = "🤖 Botlarim"
B_TOPUP = "💳 Pul kiritish"
B_ACC = "👛 Hisobim"
B_REF = "🗣️ Referal"
B_GUIDE = "📖 Qo'llanma"
B_SUP = "☎️ Qo'llab-quvvatlash"


# ========================================================
#  YORDAMCHI FUNKSIYALAR
# ========================================================
def fmt(n):
    return f"{int(n):,}".replace(",", " ")


async def safe_edit(cb: CallbackQuery, text, markup=None):
    try:
        await cb.message.edit_text(text, reply_markup=markup)
    except Exception:
        await cb.message.answer(text, reply_markup=markup)


def cancel_kb():
    kb = InlineKeyboardBuilder()
    kb.button(text="❌ Bekor qilish", callback_data="cancel")
    return kb.as_markup()


async def track(bot_id, user):
    await bot_users.update_one(
        {"bot_id": bot_id, "user_id": user.id},
        {"$set": {"first_name": user.first_name}},
        upsert=True,
    )


def menu_keyboard(doc):
    if doc["engine"] != "menu" or not doc["buttons"]:
        return None
    kb = ReplyKeyboardBuilder()
    for b in doc["buttons"]:
        kb.button(text=b["title"])
    kb.adjust(2)
    return kb.as_markup(resize_keyboard=True)
```

```
async def send_movie(message: Message, m):
    cap = m.get("caption") or None
    if m["kind"] == "video":
        await message.answer_video(m["file_id"], caption=cap)
    elif m["kind"] == "animation":
        await message.answer_animation(m["file_id"], caption=cap)
    else:
        await message.answer_document(m["file_id"], caption=cap)


# ========================================================
#  YARATILGAN BOTLARNI ISHGA TUSHIRISH / TO'XTATISH
# ========================================================
async def run_child(bot: Bot, dp: Dispatcher, bot_id: str):
    try:
        await dp.start_polling(bot, handle_signals=False)
    except asyncio.CancelledError:
        pass
    except Exception:
        logging.exception("Bot to'xtadi: %s", bot_id)
    finally:
        running.pop(bot_id, None)


async def start_child(doc):
    bot_id = str(doc["_id"])
    if bot_id in running or doc["engine"] == "soon":
        return
    try:
        token = fernet.decrypt(doc["token_enc"].encode()).decode()
    except Exception:
        logging.error("Tokenni ochib bo'lmadi: %s", bot_id)
        return
    bot = Bot(token)
    if doc["engine"] == "builder":
        dp = make_builder_dispatcher(bot_id, doc["owner_id"])
    else:
        dp = make_child_dispatcher(bot_id)
    task = asyncio.create_task(run_child(bot, dp, bot_id))
    running[bot_id] = (bot, task)


async def stop_child(bot_id: str):
    item = running.pop(bot_id, None)
    if item:
        bot, task = item
        task.cancel()
        try:
            await bot.session.close()
```

```
        except Exception:
            pass


async def delete_bot_tree(bot_id: str):
    """Botni o'chiradi. BuilderBot bo'lsa, u yaratgan botlarni ham o'chiradi."""
    for child in await bots.find({"scope": bot_id}).to_list():
        await delete_bot_tree(str(child["_id"]))
    await stop_child(bot_id)
    await bots.delete_one({"_id": ObjectId(bot_id)})
    await bot_users.delete_many({"bot_id": bot_id})
    await movies.delete_many({"bot_id": bot_id})
    await accounts.delete_many({"scope": bot_id})


# ========================================================
#  YARATILGAN ODDIY BOTLAR (kino, tugmali, murojaat)
# ========================================================
def make_child_dispatcher(bot_id: str) -> Dispatcher:
    cdp = Dispatcher()

    async def load():
        return await bots.find_one({"_id": ObjectId(bot_id)})

    @cdp.message(CommandStart())
    async def c_start(message: Message):
        doc = await load()
        if not doc:
            return
        await track(bot_id, message.from_user)
        await message.answer(doc["welcome"], reply_markup=menu_keyboard(doc))

    @cdp.message(Command("stat"))
    async def c_stat(message: Message):
        doc = await load()
        if not doc or message.from_user.id != doc["owner_id"]:
            return
        n = await bot_users.count_documents({"bot_id": bot_id})
        text = f"📊 Foydalanuvchilar: {n}"
        if doc["engine"] == "movies":
            text += f"\n🎞 Fayllar: {await movies.count_documents({'bot_id':
bot_id})}"
        await message.answer(text)

    @cdp.message(Command("xabar"))
    async def c_broadcast(message: Message, command: CommandObject):
        doc = await load()
        if not doc or message.from_user.id != doc["owner_id"]:
            return
        if not command.args:
```

```
            await message.answer("Foydalanish: /xabar matn")
            return
        ok = bad = 0
        for u in await bot_users.find({"bot_id": bot_id}).to_list():
            try:
                await message.bot.send_message(u["user_id"], command.args)
                ok += 1
            except Exception:
                bad += 1
            await asyncio.sleep(0.05)
        await message.answer(f"📢 Yuborildi: {ok}, yetib bormadi: {bad}")

    @cdp.message(Command("ochir"))
    async def c_delete_movie(message: Message, command: CommandObject):
        doc = await load()
        if not doc or doc["engine"] != "movies" or message.from_user.id !=
doc["owner_id"]:
            return
        if not command.args or not command.args.strip().isdigit():
            await message.answer("Foydalanish: /ochir 5")
            return
        r = await movies.delete_one({"bot_id": bot_id, "code_n":
int(command.args.strip())})
        await message.answer("🗑️ O'chirildi." if r.deleted_count else "Bunday kod
yo'q.")

    @cdp.message()
    async def c_any(message: Message):
        doc = await load()
        if not doc:
            return
        uid = message.from_user.id
        owner = uid == doc["owner_id"]
        eng = doc["engine"]
        await track(bot_id, message.from_user)

        if eng == "movies":
            media = message.video or message.document or message.animation
            if owner and media:
                kind = "video" if message.video else ("animation" if
message.animation else "document")
                last = await movies.find_one({"bot_id": bot_id}, sort=[("code_n",
-1)])
                n = (last["code_n"] + 1) if last else 1
                await movies.insert_one(
                    {
                        "bot_id": bot_id,
                        "code_n": n,
                        "kind": kind,
                        "file_id": media.file_id,
```

```
                        "caption": (message.caption or "")[:1000],
                    }
                )
                await message.answer(f"✅ Saqlandi. Kod: {n}\nFoydalanuvchi shu
raqamni yozsa, fayl chiqadi.")
                return
            text = (message.text or "").strip()
            if text.isdigit():
                m = await movies.find_one({"bot_id": bot_id, "code_n": int(text)})
                if m:
                    await send_movie(message, m)
                else:
                    await message.answer("❌ Bunday kod topilmadi.")
                return
            await message.answer(doc["welcome"])
            return

        if eng == "menu":
            text = message.text or ""
            for b in doc["buttons"]:
                if b["title"] == text:
                    await message.answer(b["reply"])
                    return
            await message.answer("Iltimos, menyudan tanlang 👇",
reply_markup=menu_keyboard(doc))
            return

        if eng == "contact":
            if owner:
                if message.reply_to_message:
                    m = await msg_map.find_one(
                        {"key": "contact:" + bot_id, "msg_id":
message.reply_to_message.message_id}
                    )
                    if m:
                        await message.copy_to(m["user_id"])
                        await message.answer("✅ Yuborildi")
                        return
                await message.answer("Javob berish uchun foydalanuvchi xabariga reply
qiling.")
                return
            try:
                fwd = await message.forward(doc["owner_id"])
                await msg_map.insert_one({"key": "contact:" + bot_id, "msg_id":
fwd.message_id, "user_id": uid})
                await message.answer("✅ Xabaringiz yuborildi. Tez orada javob
beramiz.")
            except Exception:
                await message.answer("Xatolik. Keyinroq urinib ko'ring.")
```

```
    return cdp


# ======================================================
#  BOT YARATUVCHI (asosiy bot va BuilderBot'lar)
# ======================================================
class Create(StatesGroup):
    token = State()


class Topup(StatesGroup):
    amount = State()
    receipt = State()


class Support(StatesGroup):
    msg = State()


class EditWelcome(StatesGroup):
    text = State()


class AddButton(StatesGroup):
    title = State()
    reply = State()


def main_kb():
    kb = ReplyKeyboardBuilder()
    for t in (B_CREATE, B_MY, B_TOPUP, B_ACC, B_REF, B_GUIDE, B_SUP):
        kb.button(text=t)
    kb.adjust(2, 2, 2, 1)
    return kb.as_markup(resize_keyboard=True)


def make_builder_dispatcher(scope: str, admin_id: int) -> Dispatcher:
    bdp = Dispatcher()
    price = BOT_PRICE if scope == "root" else 0

    # ---------- yordamchilar ----------
    async def get_card():
        s = await settings.find_one({"_id": scope})
        if s and s.get("card"):
            return s["card"]
        if scope == "root" and CARD_INFO:
            return CARD_INFO
        return "To'lov rekvizitlari hali kiritilmagan. Admin bilan bog'laning."

    async def balance_of(uid):
```

```
        acc = await accounts.find_one({"scope": scope, "user_id": uid})
        return (acc or {}).get("balance", 0)

    async def owned(bot_id, uid):
        try:
            oid = ObjectId(bot_id)
        except Exception:
            return None
        return await bots.find_one({"_id": oid, "scope": scope, "owner_id": uid})

    async def panel(doc):
        bid = str(doc["_id"])
        eng = doc["engine"]
        n = await bot_users.count_documents({"bot_id": bid})
        lines = [doc["title"], f"🔗 @{doc['username']}", f"👥 Foydalanuvchilar: {n}"]
        kb = InlineKeyboardBuilder()
        kb.button(text="✏️ Salomlashish matni", callback_data=f"wl:{bid}")
        if eng == "menu":
            titles = "\n".join(f"• {b['title']}" for b in doc["buttons"]) or "yo'q"
            lines.append(f"🔘 Tugmalar:\n{titles}")
            kb.button(text="➕ Tugma qo'shish", callback_data=f"ab:{bid}")
            kb.button(text="🗑️ Tugma o'chirish", callback_data=f"db:{bid}")
        elif eng == "movies":
            cnt = await movies.count_documents({"bot_id": bid})
            lines.append(f"🎞️ Fayllar: {cnt}")
            lines.append("➕ Kino qo'shish: botingizga o'zingiz video yuboring, bot
kod beradi.")
        elif eng == "contact":
            lines.append("📩 Foydalanuvchi xabarlari sizga keladi. Javob berish uchun
xabarga reply qiling.")
        elif eng == "builder":
            c = await bots.count_documents({"scope": bid})
            lines.append(f"🏗️ Shu bot orqali yaratilgan botlar: {c}")
        lines.append("📣 Barchaga xabar: botingizda /xabar matn yozing. Statistika:
/stat")
        lines.append(f"\n💬 Salomlashish matni:\n{doc['welcome']}")
        kb.button(text="❌ Botni o'chirish", callback_data=f"xd:{bid}")
        kb.button(text="⬅️ Orqaga", callback_data="my")
        kb.adjust(1)
        return "\n".join(lines), kb.as_markup()

    async def my_list(uid):
        docs = await bots.find({"scope": scope, "owner_id": uid}).to_list()
        if not docs:
            return "Sizda hali bot yo'q. «➕ Bot yaratish» ni bosing.", None
        kb = InlineKeyboardBuilder()
        for d in docs:
            kb.button(text=f"{d['title']} @{d['username']}", callback_data=f"mb:
{d['_id']}")
        kb.adjust(1)
```

```
        return "🤖 Botlaringiz:", kb.as_markup()

    # ---------- /start, /cancel ----------
    @bdp.message(CommandStart())
    async def start(message: Message, command: CommandObject, state: FSMContext):
        await state.clear()
        u = message.from_user
        res = await accounts.update_one(
            {"scope": scope, "user_id": u.id},
            {"$setOnInsert": {"balance": 0, "referred_by": None, "first_name":
u.first_name, "joined": time.time()}},
            upsert=True,
        )
        if res.upserted_id is not None and command.args and
command.args.startswith("ref"):
            try:
                ref_id = int(command.args[3:])
            except ValueError:
                ref_id = 0
            if ref_id and ref_id != u.id:
                ref = await accounts.find_one({"scope": scope, "user_id": ref_id})
                if ref:
                    await accounts.update_one(
                        {"scope": scope, "user_id": u.id}, {"$set": {"referred_by":
ref_id}}
                    )
                    note = "🎉 Sizning havolangiz orqali yangi foydalanuvchi
qo'shildi!"
                    if REFERAL_BONUS > 0 and scope == "root":
                        await accounts.update_one(
                            {"scope": scope, "user_id": ref_id}, {"$inc": {"balance":
REFERAL_BONUS}}
                        )
                        note += f"\n💰 Bonus: {fmt(REFERAL_BONUS)} so'm"
                    try:
                        await message.bot.send_message(ref_id, note)
                    except Exception:
                        pass
        await message.answer(
            f"Salom, {u.first_name}! 👋\nQuyidagi menyudan tanlang:",
reply_markup=main_kb()
        )

    @bdp.message(Command("cancel"))
    async def cancel_cmd(message: Message, state: FSMContext):
        await state.clear()
        await message.answer("Bekor qilindi.", reply_markup=main_kb())

    @bdp.callback_query(F.data == "cancel")
    async def cancel_cb(cb: CallbackQuery, state: FSMContext):
```

```
        await state.clear()
        await safe_edit(cb, "Bekor qilindi.")
        await cb.answer()

    # ---------- admin buyruqlari ----------
    @bdp.message(Command("stat"), F.from_user.id == admin_id)
    async def admin_stat(message: Message):
        a = await accounts.count_documents({"scope": scope})
        b = await bots.count_documents({"scope": scope})
        await message.answer(f"📊 Foydalanuvchilar: {a}\n🤖 Yaratilgan botlar:
{b}\n▶️ Ishlayotgan botlar (jami): {len(running)}")

    @bdp.message(Command("setcard"), F.from_user.id == admin_id)
    async def admin_setcard(message: Message, command: CommandObject):
        if not command.args:
            await message.answer("Foydalanish: /setcard 8600 **** **** **** Ism
Familiya")
            return
        await settings.update_one({"_id": scope}, {"$set": {"card": command.args}},
upsert=True)
        await message.answer("✅ Karta ma'lumoti saqlandi.")

    @bdp.message(Command("addbalance"), F.from_user.id == admin_id)
    async def admin_addbalance(message: Message, command: CommandObject):
        parts = (command.args or "").split()
        if len(parts) != 2 or not parts[0].lstrip("-").isdigit() or not
parts[1].lstrip("-").isdigit():
            await message.answer("Foydalanish: /addbalance USER_ID SUMMA")
            return
        await accounts.update_one(
            {"scope": scope, "user_id": int(parts[0])}, {"$inc": {"balance":
int(parts[1])}}, upsert=True
        )
        await message.answer("✅ Balans yangilandi.")

    # ---------- ASOSIY MENYU TUGMALARI ----------
    @bdp.message(F.text == B_CREATE)
    async def m_create(message: Message, state: FSMContext):
        await state.clear()
        kb = InlineKeyboardBuilder()
        kb.button(text="🤖 Telegram Bot", callback_data="k:tg")
        kb.button(text="🏗 BuilderBot", callback_data="k:b")
        kb.button(text="❌ Bekor qilish", callback_data="cancel")
        kb.adjust(2, 1)
        await message.answer(
            "🤖 Nima yaratmoqchisiz?\n\n"
            "🤖 Telegram Bot — savdo, kino, SMM va boshqa tayyor botlar.\n"
            "🏗 BuilderBot — o'z foydalanuvchilaringizga bot yaratib beruvchi shaxsiy
bot.\n\n"
            "Birini tanlang: 👇",
```

```
            reply_markup=kb.as_markup(),
        )

    @bdp.message(F.text == B_MY)
    async def m_my(message: Message, state: FSMContext):
        await state.clear()
        text, markup = await my_list(message.from_user.id)
        await message.answer(text, reply_markup=markup)

    @bdp.message(F.text == B_TOPUP)
    async def m_topup(message: Message, state: FSMContext):
        await state.clear()
        if not admin_id:
            await message.answer("To'lov tizimi hali sozlanmagan.")
            return
        await state.set_state(Topup.amount)
        await message.answer(
            f"💳 Hisobni to'ldirish\n\n{await get_card()}\n\n"
            "Qancha so'm kiritmoqchisiz? Faqat raqam yozing (masalan: 50000).\n"
            "Bekor qilish: /cancel"
        )

    @bdp.message(F.text == B_ACC)
    async def m_acc(message: Message, state: FSMContext):
        await state.clear()
        uid = message.from_user.id
        n = await bots.count_documents({"scope": scope, "owner_id": uid})
        invited = await accounts.count_documents({"scope": scope, "referred_by":
uid})
        await message.answer(
            f"👛 Hisobim\n\n🆔 ID: {uid}\n💰 Balans: {fmt(await balance_of(uid))}
so'm\n"
            f"🤖 Botlaringiz: {n}\n👥 Takliflar: {invited}"
        )

    @bdp.message(F.text == B_REF)
    async def m_ref(message: Message, state: FSMContext):
        await state.clear()
        me = await message.bot.get_me()
        uid = message.from_user.id
        invited = await accounts.count_documents({"scope": scope, "referred_by":
uid})
        text = f"🗣 Referal\n\nSizning havolangiz:\nhttps://t.me/{me.username}?
start=ref{uid}\n\n👥 Taklif qilinganlar: {invited}"
        if REFERAL_BONUS > 0 and scope == "root":
            text += f"\n💰 Har bir taklif uchun: {fmt(REFERAL_BONUS)} so'm"
        await message.answer(text)

    @bdp.message(F.text == B_GUIDE)
    async def m_guide(message: Message, state: FSMContext):
```

```
        await state.clear()
        await message.answer(
            "📖 Qo'llanma\n\n"
            "1) «➕ Bot yaratish» ni bosing.\n"
            "2) Bot turini tanlang.\n"
            "3) Telegramda @BotFather ga kirib /newbot yuboring, yangi bot oching va
u bergan TOKENni shu yerga yuboring.\n"
            "4) «🤖 Botlarim» bo'limidan botingizni boshqaring: matn, tugmalar,
statistika.\n\n"
            "🎬 Kino botlar: botingizga o'zingiz video yuboring, bot kod beradi.
Foydalanuvchi shu raqamni yozsa, video chiqadi.\n"
            "📣 Barchaga xabar yuborish: o'z botingizda /xabar matn deb yozing.\n"
            "🧱 BuilderBot: sizning foydalanuvchilaringiz ham shu tarzda o'z botini
yaratadi."
        )

    @bdp.message(F.text == B_SUP)
    async def m_sup(message: Message, state: FSMContext):
        await state.clear()
        if not admin_id:
            await message.answer("Qo'llab-quvvatlash hali sozlanmagan.")
            return
        await state.set_state(Support.msg)
        await message.answer("☎️ Savolingizni yozing, admin javob beradi.\nBekor
qilish: /cancel")

    # ---------- Bot yaratish: tanlov ----------
    @bdp.callback_query(F.data == "k:tg")
    async def k_tg(cb: CallbackQuery):
        kb = InlineKeyboardBuilder()
        for i, (title, _) in enumerate(CATEGORIES):
            kb.button(text=title, callback_data=f"cat:{i}")
        kb.button(text="❌ Bekor qilish", callback_data="cancel")
        kb.adjust(2)
        await safe_edit(cb, "🗂 Quyidagi bo'limlardan birini tanlang:",
kb.as_markup())
        await cb.answer()

    @bdp.callback_query(F.data.startswith("cat:"))
    async def k_cat(cb: CallbackQuery):
        i = int(cb.data.split(":")[1])
        title, keys = CATEGORIES[i]
        kb = InlineKeyboardBuilder()
        for k in keys:
            t = TEMPLATES[k]
            label = t["title"] + (" ⏳" if t["engine"] == "soon" else "")
            kb.button(text=label, callback_data=f"tpl:{k}")
        kb.button(text="⬅️ Orqaga", callback_data="k:tg")
        kb.button(text="❌ Bekor qilish", callback_data="cancel")
        kb.adjust(2)
```

```
        await safe_edit(cb, f"📋 {title}\nQuyidagi botlardan birini tanlang:\n(⏳ =
tez orada)", kb.as_markup())
        await cb.answer()

    async def pick_template(cb: CallbackQuery, key: str, state: FSMContext):
        t = TEMPLATES[key]
        if t["engine"] == "soon":
            await cb.answer("⏳ Bu bot tez orada qo'shiladi", show_alert=True)
            return
        uid = cb.from_user.id
        if await bots.count_documents({"scope": scope, "owner_id": uid}) >= MAX_BOTS:
            await cb.answer(f"Maksimal {MAX_BOTS} ta bot yaratish mumkin",
show_alert=True)
            return
        if price > 0 and await balance_of(uid) < price:
            await cb.answer(f"Hisobingizda mablag' yetarli emas. Bot narxi:
{fmt(price)} so'm", show_alert=True)
            return
        await state.set_state(Create.token)
        await state.update_data(key=key)
        extra = f"\n💰 Narxi: {fmt(price)} so'm" if price > 0 else ""
        await safe_edit(
            cb,
            f"{t['title']}{extra}\n\n"
            "1) Telegramda @BotFather ga kiring\n"
            "2) /newbot yuboring va yangi bot yarating\n"
            "3) U bergan TOKENni shu yerga yuboring",
            cancel_kb(),
        )
        await cb.answer()

    @bdp.callback_query(F.data.startswith("tpl:"))
    async def k_tpl(cb: CallbackQuery, state: FSMContext):
        key = cb.data.split(":")[1]
        if key in TEMPLATES:
            await pick_template(cb, key, state)

    @bdp.callback_query(F.data == "k:b")
    async def k_builder(cb: CallbackQuery, state: FSMContext):
        await pick_template(cb, "builder", state)

    # ---------- Token qabul qilish va bot yaratish ----------
    @bdp.message(Create.token, F.text)
    async def got_token(message: Message, state: FSMContext):
        data = await state.get_data()
        key = data.get("key")
        uid = message.from_user.id
        token = message.text.strip()
        try:
            await message.delete()  # xavfsizlik uchun token yozilgan xabarni
```

```
o'chiramiz
        except Exception:
            pass
        if key not in TEMPLATES:
            await state.clear()
            await message.answer("Xatolik. Qaytadan boshlang.",
reply_markup=main_kb())
            return
        try:
            probe = Bot(token)
            me = await probe.get_me()
            await probe.session.close()
        except Exception:
            await message.answer("❌ Token noto'g'ri. Qayta yuboring yoki /cancel
bosing.")
            return
        if await bots.find_one({"tg_id": me.id}):
            await state.clear()
            await message.answer("Bu bot allaqachon qo'shilgan.",
reply_markup=main_kb())
            return
        if price > 0:
            r = await accounts.update_one(
                {"scope": scope, "user_id": uid, "balance": {"$gte": price}},
{"$inc": {"balance": -price}}
            )
            if r.modified_count == 0:
                await state.clear()
                await message.answer("Hisobingizda mablag' yetarli emas.",
reply_markup=main_kb())
                return
        t = TEMPLATES[key]
        res = await bots.insert_one(
            {
                "scope": scope,
                "owner_id": uid,
                "tg_id": me.id,
                "username": me.username,
                "token_enc": fernet.encrypt(token.encode()).decode(),
                "engine": t["engine"],
                "template": key,
                "title": t["title"],
                "welcome": t["welcome"],
                "buttons": [dict(b) for b in MENU_BTNS] if t["engine"] == "menu" else
[],
                "created": time.time(),
            }
        )
        doc = await bots.find_one({"_id": res.inserted_id})
        await start_child(doc)
```

```
        await state.clear()
        text, markup = await panel(doc)
        await message.answer(f"✅ @{me.username} tayyor va ishlayapti!\n\n{text}",
reply_markup=markup)

    # ---------- Botlarim ----------
    @bdp.callback_query(F.data == "my")
    async def cb_my(cb: CallbackQuery, state: FSMContext):
        await state.clear()
        text, markup = await my_list(cb.from_user.id)
        await safe_edit(cb, text, markup)
        await cb.answer()

    @bdp.callback_query(F.data.startswith("mb:"))
    async def cb_open(cb: CallbackQuery, state: FSMContext):
        await state.clear()
        doc = await owned(cb.data.split(":")[1], cb.from_user.id)
        if not doc:
            await cb.answer("Bot topilmadi", show_alert=True)
            return
        text, markup = await panel(doc)
        await safe_edit(cb, text, markup)
        await cb.answer()

    def back_kb(bot_id):
        kb = InlineKeyboardBuilder()
        kb.button(text="⬅️ Botga qaytish", callback_data=f"mb:{bot_id}")
        return kb.as_markup()

    # salomlashish matni
    @bdp.callback_query(F.data.startswith("wl:"))
    async def wl_start(cb: CallbackQuery, state: FSMContext):
        bot_id = cb.data.split(":")[1]
        if not await owned(bot_id, cb.from_user.id):
            await cb.answer("Bot topilmadi", show_alert=True)
            return
        await state.set_state(EditWelcome.text)
        await state.update_data(bot_id=bot_id)
        await safe_edit(cb, "Yangi salomlashish matnini yuboring:", cancel_kb())
        await cb.answer()

    @bdp.message(EditWelcome.text, F.text)
    async def wl_save(message: Message, state: FSMContext):
        data = await state.get_data()
        doc = await owned(data["bot_id"], message.from_user.id)
        if doc:
            await bots.update_one({"_id": doc["_id"]}, {"$set": {"welcome":
message.text[:1000]}})
        await state.clear()
        await message.answer("✅ Saqlandi.", reply_markup=back_kb(data["bot_id"]))
```

```
    # tugma qo'shish
    @bdp.callback_query(F.data.startswith("ab:"))
    async def ab_start(cb: CallbackQuery, state: FSMContext):
        bot_id = cb.data.split(":")[1]
        doc = await owned(bot_id, cb.from_user.id)
        if not doc:
            await cb.answer("Bot topilmadi", show_alert=True)
            return
        if len(doc["buttons"]) >= 10:
            await cb.answer("Maksimal 10 ta tugma", show_alert=True)
            return
        await state.set_state(AddButton.title)
        await state.update_data(bot_id=bot_id)
        await safe_edit(cb, "Tugma nomini yozing (masalan: Narxlar):", cancel_kb())
        await cb.answer()

    @bdp.message(AddButton.title, F.text)
    async def ab_title(message: Message, state: FSMContext):
        await state.update_data(title=message.text[:30])
        await state.set_state(AddButton.reply)
        await message.answer("Bu tugma bosilganda bot nima deb javob bersin?")

    @bdp.message(AddButton.reply, F.text)
    async def ab_reply(message: Message, state: FSMContext):
        data = await state.get_data()
        doc = await owned(data["bot_id"], message.from_user.id)
        if doc:
            await bots.update_one(
                {"_id": doc["_id"]},
                {"$push": {"buttons": {"title": data["title"], "reply":
message.text[:1000]}}},
            )
        await state.clear()
        await message.answer("✅ Tugma qo'shildi.",
reply_markup=back_kb(data["bot_id"]))

    # tugma o'chirish
    @bdp.callback_query(F.data.startswith("db:"))
    async def db_list(cb: CallbackQuery):
        bot_id = cb.data.split(":")[1]
        doc = await owned(bot_id, cb.from_user.id)
        if not doc:
            await cb.answer("Bot topilmadi", show_alert=True)
            return
        if not doc["buttons"]:
            await cb.answer("Tugmalar yo'q", show_alert=True)
            return
        kb = InlineKeyboardBuilder()
        for i, b in enumerate(doc["buttons"]):
```

```
            kb.button(text=f"🗑️ {b['title']}", callback_data=f"rb:{bot_id}:{i}")
        kb.button(text="⬅️ Orqaga", callback_data=f"mb:{bot_id}")
        kb.adjust(1)
        await safe_edit(cb, "Qaysi tugmani o'chiramiz?", kb.as_markup())
        await cb.answer()

    @bdp.callback_query(F.data.startswith("rb:"))
    async def rb_do(cb: CallbackQuery):
        _, bot_id, idx = cb.data.split(":")
        doc = await owned(bot_id, cb.from_user.id)
        if not doc:
            await cb.answer("Bot topilmadi", show_alert=True)
            return
        buttons = doc["buttons"]
        i = int(idx)
        if 0 <= i < len(buttons):
            buttons.pop(i)
            await bots.update_one({"_id": doc["_id"]}, {"$set": {"buttons":
buttons}})
        doc = await owned(bot_id, cb.from_user.id)
        text, markup = await panel(doc)
        await safe_edit(cb, text, markup)
        await cb.answer("O'chirildi")

    # botni o'chirish
    @bdp.callback_query(F.data.startswith("xd:"))
    async def xd_ask(cb: CallbackQuery):
        bot_id = cb.data.split(":")[1]
        doc = await owned(bot_id, cb.from_user.id)
        if not doc:
            await cb.answer("Bot topilmadi", show_alert=True)
            return
        warn = ""
        if doc["engine"] == "builder":
            warn = "\n⚠️ Bu BuilderBot orqali yaratilgan barcha botlar ham o'chadi!"
        kb = InlineKeyboardBuilder()
        kb.button(text="✅ Ha, o'chirish", callback_data=f"xy:{bot_id}")
        kb.button(text="⬅️ Yo'q", callback_data=f"mb:{bot_id}")
        kb.adjust(1)
        await safe_edit(cb, f"Botni butunlay o'chirasizmi?{warn}", kb.as_markup())
        await cb.answer()

    @bdp.callback_query(F.data.startswith("xy:"))
    async def xy_do(cb: CallbackQuery):
        bot_id = cb.data.split(":")[1]
        if not await owned(bot_id, cb.from_user.id):
            await cb.answer("Bot topilmadi", show_alert=True)
            return
        await delete_bot_tree(bot_id)
        await safe_edit(cb, "🗑️ Bot o'chirildi.")
```

```
        await cb.answer()

    # ---------- Pul kiritish ----------
    @bdp.message(Topup.amount, F.text)
    async def topup_amount(message: Message, state: FSMContext):
        t = message.text.replace(" ", "")
        if not t.isdigit() or not (0 < int(t) <= 100_000_000):
            await message.answer("Iltimos, faqat raqam yozing (masalan: 50000) yoki
/cancel bosing.")
            return
        await state.update_data(amount=int(t))
        await state.set_state(Topup.receipt)
        await message.answer("Endi to'lov chekini (skrinshot) rasm qilib yuboring
📸")

    @bdp.message(Topup.receipt, F.photo | F.document)
    async def topup_receipt(message: Message, state: FSMContext):
        data = await state.get_data()
        u = message.from_user
        r = await topups.insert_one(
            {"scope": scope, "user_id": u.id, "amount": data["amount"], "status":
"pending", "ts": time.time()}
        )
        tid = str(r.inserted_id)
        kb = InlineKeyboardBuilder()
        kb.button(text="✅ Tasdiqlash", callback_data=f"tok:{tid}")
        kb.button(text="❌ Rad etish", callback_data=f"tno:{tid}")
        kb.adjust(2)
        try:
            await message.bot.send_message(
                admin_id, f"💳 Yangi to'lov so'rovi\n👤 {u.full_name} (ID: {u.id})\n
💰 {fmt(data['amount'])} so'm"
            )
            await message.copy_to(admin_id, reply_markup=kb.as_markup())
        except Exception:
            await message.answer("Adminga yuborib bo'lmadi. Keyinroq urinib
ko'ring.")
            await state.clear()
            return
        await state.clear()
        await message.answer("✅ So'rov adminga yuborildi. Tasdiqlangach hisobingiz
to'ldiriladi.", reply_markup=main_kb())

    @bdp.callback_query(F.data.startswith("tok:") | F.data.startswith("tno:"))
    async def topup_decide(cb: CallbackQuery):
        if cb.from_user.id != admin_id:
            await cb.answer("Ruxsat yo'q", show_alert=True)
            return
        action, tid = cb.data.split(":")
        ok = action == "tok"
```

```
        t = await topups.find_one_and_update(
            {"_id": ObjectId(tid), "scope": scope, "status": "pending"},
            {"$set": {"status": "ok" if ok else "no"}},
        )
        if not t:
            await cb.answer("Bu so'rov allaqachon ko'rib chiqilgan", show_alert=True)
            return
        if ok:
            await accounts.update_one(
                {"scope": scope, "user_id": t["user_id"]}, {"$inc": {"balance":
t["amount"]}}, upsert=True
            )
            note = f"✅ Hisobingiz {fmt(t['amount'])} so'mga to'ldirildi."
        else:
            note = "❌ To'lov so'rovingiz rad etildi. Savol bo'lsa, qo'llab-
quvvatlashga yozing."
        try:
            await cb.bot.send_message(t["user_id"], note)
        except Exception:
            pass
        try:
            await cb.message.edit_reply_markup(reply_markup=None)
        except Exception:
            pass
        await cb.answer("Bajarildi")

    # ---------- Qo'llab-quvvatlash ----------
    @bdp.message(Support.msg)
    async def support_msg(message: Message, state: FSMContext):
        u = message.from_user
        try:
            head = await message.bot.send_message(
                admin_id, f"📩 Murojaat\n👤 {u.full_name} (ID: {u.id})\nJavob berish
uchun shu xabarga reply qiling."
            )
            sent = await message.copy_to(admin_id)
            for mid in (head.message_id, sent.message_id):
                await msg_map.insert_one({"key": "support:" + scope, "msg_id": mid,
"user_id": u.id})
            await message.answer("✅ Xabaringiz yuborildi. Admin tez orada javob
beradi.", reply_markup=main_kb())
        except Exception:
            await message.answer("Yuborib bo'lmadi. Keyinroq urinib ko'ring.")
        await state.clear()

    @bdp.message(F.reply_to_message, F.from_user.id == admin_id)
    async def admin_reply(message: Message):
        m = await msg_map.find_one(
            {"key": "support:" + scope, "msg_id":
message.reply_to_message.message_id}
```

```
        )
        if m:
            try:
                await message.copy_to(m["user_id"])
                await message.answer("✅ Javob yuborildi.")
            except Exception:
                await message.answer("Yuborib bo'lmadi.")

    # ---------- Boshqa hamma xabarlar ----------
    @bdp.message()
    async def fallback(message: Message):
        await message.answer("Menyudan tanlang 👇", reply_markup=main_kb())

    return bdp


# ================================================
#  ISHGA TUSHIRISH
# ================================================
async def main():
    mother = Bot(BOT_TOKEN)
    root = make_builder_dispatcher("root", ADMIN_ID)
    # Oldin yaratilgan barcha botlarni qayta ishga tushiramiz
    for doc in await bots.find({}).to_list():
        await start_child(doc)
    await root.start_polling(mother)


if __name__ == "__main__":
    asyncio.run(main())
```